FEU INSTITUTE OF TECHNOLOGY

COLLEGE OF COMPUTER STUDIES AND MULTIMEDIA ARTS

IT0049
(WEB SYSTEM TECHNOLOGIES)

MIDTERM PROJECT

THE COMPLETE POINT-OF-SALE SYSTEM

Student Name / Group Name:	Aldrich S. Sy
Members (if Group):	**Name** / **Role** Johfreeh Fajardo / Member
Section:	
Professor:	

PROGRAM OUTCOME/S (PO) ADDRESSED BY THE LABORATORY EXERCISE

- Design, implement and evaluate computer-based systems or applications to meet desired needs and requirements. [PO: C]

COURSE LEARNING OUTCOME/S (CLO) ADDRESSED BY THE LABORATORY EXERCISE

- Apply advanced web development principles in creating, debugging, validating, and securing database-backed web applications. [CLO: 2]
- Design program flow and application structure for computing problems using CodeIgniter and Laravel frameworks. [CLO: 3]

INTENDED LEARNING OUTCOME/S (ILO) OF THE LABORATORY EXERCISE

At the end of this exercise, students must be able to:

- Integrate routing, MVC structure, database models, validated forms, file upload, and authentication into a single deployable CodeIgniter application.
- Design a relational schema connecting products, customers, staff, and transactions.
- Implement a basic sales workflow that correctly updates inventory.

BACKGROUND INFORMATION

Across Modules 1 through 4, you built a Point-of-Sale application piece by piece: pages and routing, a real database, validated forms with file upload, and authentication. This project asks you to bring all of that together into one complete, working system — and to extend it with the one piece a real POS needs that the formative assessments did not cover: recording an actual sale.

GRADING SYSTEM / RUBRIC

CRITERIA	Description
Application Architecture & Design (20 points max)	The database schema, routes, and overall program flow are well designed and appropriate for the requirements, reflecting a clear plan rather than an ad-hoc build.
Feature Completeness & Functionality (30 points max)	Product, customer, and staff management are fully functional with complete CRUD; the sales workflow correctly records a transaction, decreases stock, and prevents selling more units than are in stock.
Validation & Security (20 points max)	User input is validated; uploaded files are restricted, prepared for display when they are images, and stored safely; and management pages are protected behind authentication.

CRITERIA	Description
Code Quality & Best Practices (15 points max)	Code follows the framework's conventions, avoids duplication, and separates responsibilities cleanly across the MVC layers.
Documentation & Deployment (15 points max)	The GitHub repository is complete and well organized, the README explains setup clearly, and the hosted deployment is live, stable, and matches the submitted code.

LABORATORY ACTIVITY

Build a complete Point-of-Sale system in CodeIgniter using the schema below.

1. Design and create the database using this schema (adjust field lengths if your design reasonably requires it, but keep the relationships intact):

```
CREATE TABLE products (
  id INT AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(100) NOT NULL,
  price DECIMAL(10,2) NOT NULL,
  stock_quantity INT NOT NULL DEFAULT 0,
  image VARCHAR(255),
  created_at DATETIME NOT NULL
);

CREATE TABLE customers (
  id INT AUTO_INCREMENT PRIMARY KEY,
  full_name VARCHAR(100) NOT NULL,
  email VARCHAR(100) NOT NULL,
  phone VARCHAR(20),
  created_at DATETIME NOT NULL
);

CREATE TABLE users (
  id INT AUTO_INCREMENT PRIMARY KEY,
  username VARCHAR(50) NOT NULL UNIQUE,
  full_name VARCHAR(100) NOT NULL,
  password VARCHAR(255) NOT NULL,
  avatar VARCHAR(255),
  created_at DATETIME NOT NULL
);

CREATE TABLE sales (
  id INT AUTO_INCREMENT PRIMARY KEY,
  product_id INT NOT NULL,
  customer_id INT,
```

```
  sold_by INT NOT NULL,
  quantity INT NOT NULL,
  total_price DECIMAL(10,2) NOT NULL,
  created_at DATETIME NOT NULL,
  FOREIGN KEY (product_id) REFERENCES products(id),
  FOREIGN KEY (customer_id) REFERENCES customers(id),
  FOREIGN KEY (sold_by) REFERENCES users(id)
);
```

2. Implement Product management: list, add, edit, and delete (or archive) products, including an uploaded product image prepared for display.
3. Implement Customer management with full CRUD.
4. Implement Staff (user) management with full CRUD, including avatar upload, display-ready avatar preparation, and hashed passwords.
5. Implement authentication so that only a logged-in staff member can reach any management page.
6. Implement a Record Sale page: a logged-in staff member selects a product and, optionally, a customer, enters a quantity, and submits. The product's stock_quantity must decrease by the sold quantity, and the sale must be rejected with a clear message if the requested quantity exceeds available stock.
7. Implement a Sales History page listing past transactions with product, customer (if any), staff member, quantity, total price, and date.

Submission Requirements:

- A link to a GitHub repository containing your raw project files, including your database export or migration and seeder files.
- A link to your hosted, working version of the application.

REFERENCES

- Manfield, A. (2021). Pro CodeIgniter: Learn how to create professional web-applications with PHP.
- CodeIgniter User Guide. Models. https://codeigniter4.github.io/userguide/models/model.html
- CodeIgniter User Guide. Validation. https://codeigniter4.github.io/userguide/libraries/validation.html
- CodeIgniter User Guide. Controller Filters. https://codeigniter4.github.io/userguide/incoming/filters.html